Validação de segurança antes do deploy

Sep 30, 2026 · @Nickolas

Resultado

Aprovado para o deploy de teste, com 6 ações suas na configuração dos serviços (seção "Ações suas antes do deploy"). Nenhuma falha crítica ficou aberta no código. Quatro pontos foram corrigidos nesta validação e estão no pacote `etapa13-seguranca.zip`.

O que foi executado:

Verificação	Ferramenta	Resultado
Testes do banco, com 57 testes de segurança (estrutura e ataques simulados)	pgTAP	355 de 355 passando
Testes do pacote compartilhado, do worker e do app	Node test runner	43, 30 e 6 passando
Segredos no código	gitleaks 8.28 + busca por padrões (`sk-ant-` , `sb_secret_` , `sk_live_` , `whsec_` , JWT)	Nenhum segredo encontrado
Vulnerabilidades nas dependências	`pnpm audit` (produção e desenvolvimento)	Nenhuma vulnerabilidade conhecida
Segredos no código enviado ao navegador	Build de produção com chave falsa, busca nos arquivos públicos	Nenhuma chave de servidor no navegador
Rotas privadas sem login	`curl` no app rodando em modo de produção	Todas redirecionam para Entrar
Cabeçalhos de segurança	`curl -I` no app em produção	CSP com nonce, HSTS e os demais presentes
Calendário com token inválido	`curl`	404, sem gravar nada
Tipos	TypeScript (`tsc --noEmit`)	Sem erros

Checklist completo

Itens repetidos na sua lista (rate limit, validação, dependências, secrets) aparecem uma vez. "PMP" foi lido como **princípio do menor privilégio**.

Item	Status	Como está e qual a evidência
Esconder chaves de API (Secrets)	OK	Chaves de servidor só em variáveis sem `NEXT_PUBLIC_`; o cliente administrativo importa `server-only` e quebra o build se usado no navegador; build com chave falsa não a expôs; `.env` no `.gitignore`
Varredura de segredos no Git	OK	gitleaks sem achados; no CI, gitleaks roda em todo o histórico a cada envio
RLS ligada	OK	Teste confirma RLS em todas as tabelas do `public`; o schema `private` não é acessível pela API
Controle de acesso	OK	Papéis (titular, administrador, membro) conferidos no banco; ações sensíveis em funções que validam o papel
Menor privilégio (PMP)	OK	Sem escrita direta em tabelas; UPDATE só em colunas de perfil e dados do escritório; funções do worker, Stripe e avisos só para `service_role`; sem login, só 4 funções públicas previstas
Testar IDOR / acesso a registros	OK	17 ataques simulados com ids de outro escritório (ler, editar, reprocessar, marcar, versionar): todos bloqueados
Isolamento entre usuários	OK	Cada escritório só vê os próprios dados; testado em todas as 19 suítes do banco
Rotas administrativas	OK	`/admin` exige registro em `platform_admins` e sessão com 2FA (`aal2`), conferidos no banco a cada consulta; para os demais, "página não encontrada"
Rotas sem login	OK	O proxy redireciona tudo que não é público; testado com `curl`, inclusive as APIs

Item	Status	Como está e qual a evidência
Rate limit	Corrigido	Banco: limites por usuário em envios, convites, clientes, modelos, minutas, edições, prazos e avaliações. Agora também visualizações e downloads, exportação, download `.ics` e calendário por token. Login, cadastro e e-mails: limites do Supabase Auth
Storage	OK	Bucket privado, 25 MB e só PDF/DOCX; usuário não lista arquivos nem grava fora do próprio caminho; download só por URL assinada de 5 minutos
Validação de entradas (frontend e backend)	OK	Frontend: campos obrigatórios, tipos, tamanhos e regra de senha. Backend: Zod nas ações e restrições (`check`) no banco. O backend nunca confia no frontend
Sanitização	OK	Busca com `%` e `_` agora literal (corrigido); textos gravados como texto; React escapa toda saída; e-mails escapam conteúdo; nada usa `dangerouslySetInnerHTML`; dados do usuário delimitados no prompt da IA
SQL injection	OK	Sem SQL montado com texto do usuário; testes com `'; drop table`, `OR 1=1` e afins não quebram nada
Adulteração de campos (mass assignment)	OK	12 tentativas de alterar papel, cota, plano, suspensão, status e uso: todas recusadas
Upload de arquivos	OK	Tipo, tamanho e páginas conferidos no navegador, no servidor (conteúdo real do arquivo) e no bucket; senha e arquivos corrompidos recusados
Lógica do servidor	OK	Ações do servidor revalidam sessão e permissão; regras de negócio (cota, limites, preço) no banco; rotas que alteram dados recusam outra origem
Respostas enxutas	OK	Erros só com código e mensagem própria, nunca o erro interno; consultas selecionam só as colunas usadas; painel sem conteúdo de documentos

Item	Status	Como está e qual a evidência
Sessões	Corrigido	Token de 1 hora com rotação e detecção de reuso; cookies `Secure` e `SameSite=Lax`. Agora troca e redefinição de senha encerram as outras sessões
Expiração de sessão	Ação sua	Máximo de 7 dias e 12 horas de inatividade no `config.toml`; na nuvem, exige o plano Pro (sem ele, vale só a renovação do token)
Senhas com hash	OK	Supabase Auth guarda com bcrypt; o app nunca vê nem grava senha; tokens de convite e calendário guardados só como hash SHA-256
MFA	OK	Opcional para usuários e obrigatório no painel, exigido no banco
HTTPS	OK	Vercel, Railway e Supabase só servem HTTPS; HSTS de 2 anos e `upgrade-insecure-requests`
CSP	OK	Nonce por requisição, `strict-dynamic`, `frame-ancestors 'none'`, `object-src 'none'`; conferido no cabeçalho
CORS	OK	Nenhuma rota libera outra origem; ações do servidor checam a origem; APIs de escrita recusam origem diferente
Criptografia	OK	Em trânsito por TLS; em repouso, discos do Supabase cifrados (AES-256); tokens com hash
Logs sensíveis	OK	Worker registra só ids, códigos, tempos e custos; o app não usa `console.log`; Sentry configurado para não coletar usuário, cookies, cabeçalhos e corpos
Dependências	OK	`pnpm audit` limpo; no CI, falha com vulnerabilidade alta ou crítica; Dependabot semanal
Migrations	OK	Versionadas, aplicadas em ordem e testadas a cada envio no CI

Item	Status	Como está e qual a evidência
Rollback	Ação sua	Código: voltar o deploy na Vercel e no Railway. Banco: backup antes de cada `db push` (`pnpm db:backup`, novo)
Backups	Ação sua	Plano gratuito sem backup automático; Pro com backup diário; `pnpm db:backup` para cópia manual
Monitoramento e alertas	Ação sua	Aviso de fila parada aos administradores (pronto); Sentry pronto, depende de você criar o projeto
Plano de recuperação	OK	Seção "Backup, rollback e plano de recuperação" abaixo
Atacar o próprio app	OK	57 ataques automatizados no banco, mais testes de rota com `curl`; detalhes na próxima seção

Cenários de ataque do seu pedido

Cenário	Resultado	Por quê
Logado, troca o id de um registro na requisição e acessa o de outra pessoa (IDOR)	Bloqueado	Toda leitura e escrita passa pela RLS do banco, que só devolve registros do escritório da sessão. Os testes usam ids reais do escritório A com a sessão do B, em 17 operações
Troca o id na URL (`/documentos/<id>`) e vê dados de outra pessoa	Bloqueado	A página busca com a sessão do usuário; registro de outro escritório não é encontrado e a resposta é "página não encontrada", sem confirmar que o id existe
Envia `role: "admin"` junto com nome e e-mail	Bloqueado	O banco só permite UPDATE em colunas específicas (nome, telefone, OAB, preferências). Papel muda só por função que confere se quem pede é o titular. As ações do servidor montam o objeto campo a campo, nunca repassam o que veio do formulário

Cenário	Resultado	Por quê
Altera o preço ou o plano antes de enviar	Bloqueado	O navegador envia só o identificador do plano. O servidor busca o plano no banco e o preço no Stripe pela chave de busca; valor nenhum vem do navegador
Webhook falso de "pagamento aprovado"	Bloqueado	A assinatura do evento é verificada com o segredo do webhook; sem ela, resposta 400. Mesmo com evento válido, a assinatura é relida da API do Stripe antes de gravar, e o acesso depende do status real. As funções que gravam assinatura só aceitam `service_role`
Link de redefinição de senha reutilizável ou sem prazo	Bloqueado	O link é de uso único e vale 1 hora (`otp_expiry`), controlado pelo Supabase Auth. Agora, ao redefinir, as outras sessões são encerradas
Sem Content Security Policy	Já existia	CSP com nonce novo a cada requisição; scripts injetados não executam e o app não pode ser embutido em outro site
Validação só no frontend	Já existia	Toda regra existe no servidor (Zod) e no banco (`check`, funções); o frontend repete as principais só para dar retorno imediato
Apagar ou ler arquivos de outro escritório no Storage	Bloqueado	Políticas do bucket só aceitam o caminho exato do próprio envio; listagem bloqueada; download só por URL assinada de 5 minutos, gerada após checar a permissão
Usar a chave pública do Supabase direto, fora do app	Bloqueado	A chave pública não dá acesso a nada além do que a RLS e as permissões liberam; os testes de ataque rodam exatamente nesse nível
SQL injection na busca	Bloqueado	Parâmetros sempre separados do SQL; `'; drop table documents; --` e `' OR 1=1 --` testados

Cenário	Resultado	Por quê
Força bruta no login	Limitado	Supabase Auth limita tentativas por IP; 2FA disponível
Abrir redirecionamento para outro site (`?next=//site-falso`)	Bloqueado	O destino é resolvido e só aceito se for do próprio app; testes com variações conhecidas

O que foi corrigido nesta validação

Tudo está no `etapa13-seguranca.zip` , com a migration `20261030000100_security_hardening.sql` e 7 testes novos (`019_security_hardening.test.sql`).

Ponto	Antes	Agora
Busca de clientes e do painel	`%` digitado listava todos os registros; `_` casava qualquer letra	Os dois são tratados como texto literal
Calendário assinado (endereço público)	Sem limite de leituras	240 leituras por hora por endereço; acima disso, resposta 429
Visualizações, downloads, minutas, exportação e `.ics`	Sem limite nas leituras	1.200 acessos a documentos por hora por usuário; exportação 10 por dia; `.ics` 120 por hora
Troca e redefinição de senha	Outras sessões continuavam abertas	Encerradas automaticamente
Pasta de e-mails do worker	Erro `permission denied` no modo `console`	Pasta criada com permissão no `Dockerfile`
Backup antes de migrations	Sem comando pronto	`pnpm db:backup` gera estrutura e dados em `backups/` (fora do Git)
Expiração de sessão	Só a renovação do token	Máximo de 7 dias e 12 horas sem uso no `config.toml`

Já estavam no projeto e foram conferidos: CSP com nonce e demais cabeçalhos, cookies `Secure` / `SameSite`, proteção contra redirecionamento aberto, regra de senha igual no app e no Supabase, checagem de origem nas APIs, limites por gatilho no banco, gitleaks e auditoria de dependências no CI, e Dependabot.

Ações suas antes do deploy

São configurações nos painéis dos serviços, que o código não alcança.

- [ ] **Trocar a secret key do Supabase na nuvem** e apagar a antiga (ela apareceu na nossa conversa): Project Settings → API Keys.
- [ ] **Replicar no painel do Supabase as regras de autenticação** do `config.toml`, porque o arquivo só vale para o Supabase local:
  - Authentication → Sign In / Providers → Email: **Confirm email** ligado, **Secure email change** ligado, **Secure password change** ligado;
  - Authentication → Policies (ou Passwords): mínimo de 8 caracteres e exigência de maiúsculas, minúsculas e números;
  - Authentication → Rate Limits: conferir os limites de e-mails, login e verificações;
  - Authentication → Multi-Factor: TOTP habilitado.
- [ ] **Limite de gasto na Anthropic** (Billing) e chave separada para o ambiente publicado.
- [ ] **Rodar `pnpm db:backup` antes de cada `pnpm supabase db push`** no banco da nuvem.
- [ ] **Conferir no `git status` que nenhum `.env` vai para o GitHub**, e manter o repositório privado.
- [ ] (Recomendado) **Criar o projeto no Sentry** e preencher os DSNs na Vercel e no Railway, para ser avisado de erros.

Quando for para produção de verdade, inclua também: plano Pro do Supabase (backup diário, expiração de sessão e proteção contra senhas vazadas), Vercel Pro e revisão jurídica dos termos e da política de privacidade.

Backup, rollback e plano de recuperação

Para o teste, a meta é perder no máximo os dados desde o último backup manual e voltar ao ar em até 1 hora.

Backup

- **Banco:** `pnpm db:backup` (com o projeto vinculado) gera `backups/schema.sql` (estrutura) e `backups/dados.sql` (dados). Rode antes de cada `db push` e, durante o teste, uma vez por semana. Guarde os arquivos fora do computador (um drive pessoal

com acesso restrito), porque contêm dados do teste.

- **Arquivos (Storage):** não entram no `db dump`. No teste, os originais podem ser reenviados; em produção, o plano Pro e uma rotina de cópia do bucket resolvem.
- **Código:** o GitHub é o backup; cada deploy corresponde a um commit.

Rollback

O que deu errado	Como voltar
Deploy do app com erro	Vercel → Deployments → deploy anterior → Promote to Production (segundos)
Deploy do worker com erro	Railway → Deployments → deploy anterior → Redeploy
Migration com erro	As migrations não têm "desfazer" automático. Corrigir com uma nova migration (preferível) ou restaurar o backup feito antes do `db push`
Chave vazada	Gerar uma nova no serviço, atualizar na Vercel e no Railway e apagar a antiga

Recuperação de incidentes

1. **App fora do ar:** veja os logs na Vercel e no Railway e o status do Supabase (painel). Projeto pausado por inatividade: **Restore project**.
2. **Suspeita de acesso indevido a uma conta:** em `/admin`, suspenda o escritório; peça ao usuário para redefinir a senha (o que encerra as outras sessões) e ativar o 2FA.
3. **Chave exposta:** troque-a imediatamente (tabela acima) e revise os logs de uso do serviço correspondente.
4. **Dados corrompidos:** restaure o último `backups/dados.sql` num projeto novo, confira e só então troque as variáveis para apontar para ele.
5. **Registro:** anote o que aconteceu, quando e o que foi feito. Se houver dados pessoais envolvidos, avalie com o escritório a comunicação prevista na LGPD.

Limites desta validação e próximos passos

Esta é uma revisão de código com testes automatizados e ataques simulados, suficiente para um teste fechado. Ela não substitui um teste de invasão feito por terceiros antes da produção.

- **Ambiente dos testes:** os ataques ao banco rodaram num Postgres com as mesmas migrations, e as rotas foram testadas com o app em modo de produção, mas sem o Supabase Auth real. No ambiente publicado, repita o roteiro de testes do guia de instalação, em especial os itens de isolamento (9.1) e o painel (9.12).
- **Configurações do Supabase na nuvem:** regras de senha, confirmação de e-mail e limites do Auth precisam ser conferidas no painel; o `config.toml` só vale localmente.
- **Cookies de sessão:** não são `HttpOnly`, porque o cliente do Supabase no navegador precisa lê-los (envio direto ao Storage e 2FA). A proteção contra roubo por script injetado é a CSP com nonce, que impede a execução desses scripts.
- **Senhas vazadas:** a checagem contra bases de senhas vazadas do Supabase Auth exige o plano Pro.
- **Antes da produção:** teste de invasão externo, revisão do Stripe em modo live (webhook de produção e cartões reais), DPA com os provedores (Supabase, Anthropic, Vercel, Railway, Brevo) e política de privacidade com a transferência internacional pela IA.

Para repetir esta validação a qualquer momento: `pnpm security:audit` (dependências e testes do banco) e o gitleaks, que já roda no CI a cada envio.